

# Numerische Methoden

Kompaktes Lernskript – alle Themen erklärt

Theorie + durchgerechnete Mini-Beispiele · nach Themen gegliedert

---

## Inhaltsverzeichnis

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>1 Grundlagen &amp; Zahldarstellung</b>              | <b>1</b> |
| 1.1 Warum Numerik? Diskretisierung . . . . .           | 1        |
| 1.2 Gleitkomma (floating point) . . . . .              | 2        |
| 1.3 Festkomma (fixed point) . . . . .                  | 2        |
| 1.4 Rundung, Auslöschung, Fehlerarten . . . . .        | 2        |
| 1.5 Brute Force: Grid Search . . . . .                 | 3        |
| <b>2 Fehleranalyse: die vier Eigenschaften</b>         | <b>3</b> |
| <b>3 Nullstellen: Newton- &amp; Sekantenverfahren</b>  | <b>4</b> |
| 3.1 Newton-Verfahren – Herleitung . . . . .            | 4        |
| 3.2 Numerische Ableitung & Sekantenverfahren . . . . . | 5        |
| <b>4 Globale Optimierung</b>                           | <b>5</b> |
| <b>5 Integration &amp; Interpolation</b>               | <b>6</b> |
| 5.1 Newton-Cotes-Quadratur . . . . .                   | 6        |
| 5.2 Lagrange-Interpolation . . . . .                   | 6        |
| 5.3 Romberg & Monte-Carlo . . . . .                    | 7        |
| <b>6 Modelltraining aus ersten Prinzipien</b>          | <b>7</b> |
| <b>7 Anhang: weitere Foliensatz-Themen (kompakt)</b>   | <b>8</b> |

---

## 1 Grundlagen & Zahldarstellung

### 1.1 Warum Numerik? Diskretisierung

Viele Probleme haben *keine* geschlossene Lösung (z. B.  $\int e^{-x^2} dx$ ,  $x = \cos x$ ) oder sind zu teuer analytisch. Numerik approximiert sie mit endlich vielen Rechenschritten. Dazu wird ein kontinuierliches Problem **diskretisiert**: das Intervall  $[a, b]$  in  $N$  Stücke geteilt.

#### Diskretisierung

Schrittweite  $h = \frac{b-a}{N}$ , Gitterpunkte  $x_i = a + i h$  für  $i = 0, \dots, N$  (das sind  $N+1$  Punkte).

### Beispiel

$[0, 3]$ ,  $N = 6$ :  $h = \frac{3-0}{6} = 0,5$ , Gitterpunkte 0; 0,5; 1; 1,5; 2; 2,5; 3 (sieben Stück).

## 1.2 Gleitkomma (floating point)

### Gleitkommazahl

$x = (-1)^s m 2^e$  mit Vorzeichenbit  $s$ , **Mantisse**  $m$  (Genauigkeit) und **Exponent**  $e$  (Größenordnung). *Normalisiert*:  $m = 1.b_1b_2\dots$  mit *implizitem* führenden 1-Bit. Der **Bias** verschiebt den Exponenten, damit auch negative  $e$  ohne eigenes Vorzeichen passen.

### Maschinenepsilon

$\varepsilon_{\text{mach}} = 2^{-t}$  bei  $t$  gespeicherten Nachkomma-Mantissenbits = Abstand von 1 zur nächstgrößeren darstellbaren Zahl = kleinste *relative* Auflösung.

### Der Trade-off Auflösung $\leftrightarrow$ Wertebereich

Bei fester Bitzahl gilt:

- **Ein Bit mehr Mantisse**  $\Rightarrow$  kleineres  $\varepsilon_{\text{mach}}$   $\Rightarrow$  *feinere Auflösung* (resolution). Die Zahlen werden *nicht* größer, nur genauer.
- **Ein Bit mehr Exponent**  $\Rightarrow$  *größerer Wertebereich* (dynamic range): betragsmäßig größte und kleinste Zahl wachsen. Die Genauigkeit wird *nicht* feiner.

Kurz: Mantisse = *wie genau*, Exponent = *wie weit*.

## 1.3 Festkomma (fixed point)

Feste Anzahl Vor- und Nachkommabits, Skalierungsfaktor  $2^{-f}$  ( $f$  = Nachkommabits).

### Festkomma

Auflösung =  $2^{-f}$  (*konstant*, überall gleich). Größte vorzeichenlose Zahl mit  $k$  Bit gesamt:  $\frac{2^k - 1}{2^f}$ .  
Speicherwert von  $x$ :  $x \cdot 2^f$  (ganzzahlig), dann binär.

### Beispiel

8 Bit, 4 Vor-/4 Nachkomma ( $f = 4$ ): Auflösung  $2^{-4} = 0,0625$ , größte Zahl  $\frac{2^8 - 1}{16} = 15,9375$ .  
Darstellung 5,25:  $5,25 \cdot 16 = 84 = 01010100_2$  (Probe  $0101,0100 = 4 + 1 + 0,25$ ). Vergleich:  
**Festkomma** hat *konstante absolute* Abstände; **Gleitkomma** hat *konstante relative* Abstände (absoluter Abstand wächst mit der Größe von  $x$ ).

## 1.4 Rundung, Auslöschung, Fehlerarten

### Fehlermaße

Absoluter Fehler  $|x - \tilde{x}|$ ; relativer Fehler  $\frac{|x - \tilde{x}|}{|x|}$  (skaleninvariant, „gültige Stellen“). Relativ ist meist aussagekräftiger – außer nahe  $x = 0$ .

### Auslöschung (catastrophic cancellation)

Subtrahiert man zwei *fast gleiche* Zahlen, heben sich die führenden Stellen weg; der kleine Rest ist oft schon weggerundet  $\Rightarrow$  der *relative* Fehler explodiert. Gegenmittel: umformen, sodass nicht subtrahiert wird, z. B.  $\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$ .

### Auslöschung im Mini-System

System mit 2 Mantissenbits:  $\varepsilon = 2^{-2} = 0,25$ , darstellbar 1,00; 1,25; 1,5; 1,75.  $1 + 0,1 = 1,1 \rightarrow$  rundet auf 1,0 (denn  $|1,1 - 1| = 0,1 < |1,1 - 1,25| = 0,15$ ). Dann  $\frac{1}{(1+0,1)-1} = \frac{1}{1-1} = \frac{1}{0} \rightarrow \infty$ , mathematisch wäre es 10 – die Information 0,1 ging beim Runden verloren.

### Die drei Fehlerarten

**Modellfehler** (Realität vereinfacht, z. B. Reibung weggelassen) · **Abschneide-/Diskretisierungsfehler** (Grenzprozess abgebrochen: Reihe nach  $k$  Gliedern, Ableitung  $\rightarrow$  Differenzenquotient) · **Rundungsfehler** (endliche Stellenzahl, z. B.  $\frac{1}{3} \rightarrow 0,333$ ).

## 1.5 Brute Force: Grid Search

Probiere alle Kandidaten auf einem Gitter durch und nimm den mit kleinstem Fehler.

### Grid Search für $A\mathbf{x} = \mathbf{c}$

Fehlermaß  $E(\mathbf{x}) = \|A\mathbf{x} - \mathbf{c}\|$  (z. B. Summe der Residuen-Beträge oder -Quadrate). Über das Gitter minimieren. **Wichtig:** das Residuum misst im *Ausgaberaum* ( $A\mathbf{x}$  vs.  $\mathbf{c}$ ) – nicht den Abstand der Parameter zur exakten Lösung.

### Beispiel

$\begin{pmatrix} 1 & 2 \\ 3 & 1 \end{pmatrix} \begin{pmatrix} w \\ b \end{pmatrix} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}$ , Gitter  $w, b \in \{0, 1, 2\}$ ,  $E = |w + 2b - 4| + |3w + b - 5|$ . Bestes Gitterpaar:  $(w, b) = (1, 2)$  mit  $E = 1$ . Exakt ist  $(\frac{6}{5}, \frac{7}{5})$  – also *nicht* getroffen. Grid Search trifft exakt nur, wenn die Lösung zufällig auf dem Gitter liegt; Verbesserung: feineres Gitter / um den Sieger verfeinern.

## 2 Fehleranalyse: die vier Eigenschaften

Schreibweise:  $f$  = wahres Problem,  $\hat{f}$  = Algorithmus,  $\tilde{x}$  = gestörte Eingabe.

### Die vier Begriffe

|                   |                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Kondition</b>  | $\kappa =  f(x) - f(\tilde{x}) $ – Empfindlichkeit des <i>Problems</i> auf Eingabestörung.                    |
| <b>Stabilität</b> | $s = \frac{ \hat{f}(\tilde{x}) - \hat{f}(x) }{ \tilde{x} - x }$ – verstärkt der <i>Algorithmus</i> Störungen? |
| <b>Konsistenz</b> | $c =  \hat{f}(x) - f(x) $ – wie gut nähert der Algorithmus bei exakter Eingabe?                               |
| <b>Konvergenz</b> | $ \hat{f}(\tilde{x}) - f(x)  \rightarrow 0$ – nähert sich das Resultat dem wahren Wert?                       |

### Faustregeln

$\kappa \approx |f'(x)| \cdot \delta$ : *flach* (kleine Steigung) = **gut** konditioniert, *steil* = **schlecht** konditioniert. Kondition gehört zum *Problem*, Stabilität zum *Verfahren*: ein stabiler Algorithmus kann ein schlecht konditioniertes Problem *nicht* retten. **Konvergenz**  $\approx$  **Konsistenz** + **Stabilität**.

$g(x) = (x-2)^2$ , Störung  $\delta = \frac{1}{2}$ , Gitter  $h_1=1$ ,  $h_2=\frac{1}{2}$

**Kondition:** nahe Minimum  $x=2$ :  $\kappa = |g(2) - g(2,5)| = |0 - 0,25| = 0,25$  (gut); am Rand  $x=0$ :  $\kappa = |g(0) - g(0,5)| = |4 - 2,25| = 1,75$  (schlecht).

**Konsistenz** an  $x=2,7$  ( $g(2,7) = 0,49$ ):  $h_1: 2,7 \rightarrow 3$ ,  $c_1 = |g(3) - 0,49| = 0,51$ ;  $h_2: 2,7 \rightarrow 2,5$ ,  $c_2 = |g(2,5) - 0,49| = 0,24 \Rightarrow$  feineres  $h_2$  konsistenter.

**Stabilität:** feineres Gitter  $\Rightarrow$  Störung springt eher auf einen anderen Gitterpunkt  $\Rightarrow$  tendenziell weniger stabil (referenzpunktabhängig).

**Führt kleineres  $h$  immer zu besserer Näherung?**

**Nein.** Kleineres  $h$  verbessert die *Konsistenz*, kann aber *Stabilität* verschlechtern, und ab einem Punkt dominiert der *Rundungsfehler* (bzw. das Eingangsrauschen). Es gibt ein *optimales  $h$* .

### 3 Nullstellen: Newton- & Sekantenverfahren

#### 3.1 Newton-Verfahren – Herleitung

**Herleitung aus Taylor 1. Ordnung**

$f(x) \approx f(x_n) + f'(x_n)(x - x_n)$ . Diese Tangente = 0 setzen und nach  $x$  lösen:

$$0 = f(x_n) + f'(x_n)(x_{n+1} - x_n) \Rightarrow x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Newton iteriert also die *Tangenten-Nullstelle*.

**Konvergenz**

**Kriterium:**  $x_0$  nah genug an einer *einfachen* Wurzel  $x^*$ ,  $f \in C^2$ ,  $f'(x^*) \neq 0$ . Dann **quadratische Konvergenz**:  $|e_{n+1}| \leq C |e_n|^2$  – praktisch *verdoppelt* sich die Zahl korrekter Stellen pro Schritt.

$f(x) = x^2 - 2$  ( $\sqrt{2}$ ),  $x_0 = 2$ ,  $f' = 2x$

$$x_1 = 2 - \frac{4-2}{2 \cdot 2} = 2 - \frac{1}{2} = \frac{3}{2}, \quad x_2 = \frac{3}{2} - \frac{(3/2)^2 - 2}{2 \cdot \frac{3}{2}} = \frac{3}{2} - \frac{1/4}{3} = \frac{3}{2} - \frac{1}{12} = \frac{17}{12} \approx 1,4167.$$

**Beim 2. Schritt den Nenner an der neuen Stelle bilden**

$x_2$  braucht  $f'(x_1) = f'(\frac{3}{2}) = 2 \cdot \frac{3}{2} = 3$  (nicht  $f'(x_0)$ !). Also Nenner 3, nicht etwa 4,5. Allgemein: in jeder Iteration  $f(x_n)$  und  $f'(x_n)$  am *aktuellen*  $x_n$  auswerten.

**Versagensmodi von Newton**

$f'(x_n) = 0$  (Division durch 0, waagrechte Tangente) · schlechter Startwert  $\rightarrow$  Divergenz / falsche Wurzel · *Zyklus* (Iteration pendelt). Mehrfache Wurzel  $\Rightarrow$  nur lineare Konvergenz.

## 3.2 Numerische Ableitung & Sekantenverfahren

### Vorwärtsdifferenz & Sekante

$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h}$ , Fehler  $\mathcal{O}(h)$  (Taylor-Restterm  $\frac{h}{2}f''(\xi)$ ). Ersetzt man  $f'$  durch den Differenzenquotienten der *letzten zwei* Punkte:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \quad (\text{Sekante}).$$

Ordnung  $\approx 1,618$  (superlinear). Vorteil: *ableitungsfrei*; Nachteil: braucht zwei Startwerte, Nenner kann  $\approx 0$  werden.

Ein Sekantenschritt,  $f = x^2 - 2$ ,  $x_0 = 2$ ,  $x_1 = 1,5$

$f(2) = 2$ ,  $f(1,5) = 0,25$ :

$$x_2 = 1,5 - 0,25 \cdot \frac{1,5 - 2}{0,25 - 2} = 1,5 - 0,25 \cdot \frac{2}{7} = 1,5 - \frac{1}{14} = \frac{10}{7} \approx 1,4286.$$

## 4 Globale Optimierung

### Lokal vs. global, Konvexität

**Lokales** Minimum: kleinster Wert in einer *Umgebung*; **globales**: kleinster Wert auf der *ganzen* Domäne. Lokale Verfahren (Gradientenabstieg, Newton) laufen ins *nächste* Tal und verfehlen das globale (multimodale „Foxholes“). Ist  $f$  **konvex** ( $f'' \geq 0$ ), so ist *jedes* lokale Minimum global – dann genügen lokale Verfahren.

### Newton fürs Optimum

Optimum  $\Leftrightarrow f'(x) = 0$ . Newton auf  $f'$ :  $x_{n+1} = x_n - \frac{f'(x_n)}{|f''(x_n)|}$ . Der **Betrag**  $|f''|$  erzwingt einen Schritt *immer bergab* (Richtung Minimum), auch in konkaven Bereichen. Klassifikation:  $f'' > 0 \Rightarrow$  Minimum,  $f'' < 0 \Rightarrow$  Maximum.

$g = x^3 - 3x$ ,  $x_0 = 2$

$g' = 3x^2 - 3$ ,  $g'' = 6x$ :  $x_1 = 2 - \frac{g'(2)}{|g''(2)|} = 2 - \frac{9}{12} = \frac{5}{4} = 1,25$ .  $g''(\frac{5}{4}) = 7,5 > 0 \Rightarrow$  lokales Minimum.

### Strategien gegen lokale Minima

**Random Restarts**: viele zufällige Starts, bester gewinnt; Erfolg  $P = 1 - (1-p)^K$ . **Simulated Annealing**: akzeptiere Verschlechterung mit  $e^{-\Delta E/T}$ , kühle  $T$  langsam ab (erst Exploration, dann Exploitation). **Basin Hopping**: zufälliger Sprung + lokale Minimierung. **Glättung**: hochfrequenten Störterm wegmitteln.

### Random Restarts & Glättung

$p = \frac{1}{2}$ :  $P(K=3) = 1 - (\frac{1}{2})^3 = \frac{7}{8} = 0,875$ ; kleinstes  $K$  mit  $P > 0,9$ :  $(\frac{1}{2})^K < 0,1 \Leftrightarrow 2^K > 10 \Rightarrow K = 4$ .

Rauer Term  $\frac{1}{2} \sin(4\pi x)$ : Periode  $= \frac{2\pi}{4\pi} = \frac{1}{2}$ . Wähle das Glättungs-/ Schrittfenster  $h \approx \frac{1}{2}$ , damit sich der Sinus über eine ganze Periode herausmittelt.

## Gradientenabstieg

$x_{n+1} = x_n - \eta f'(x_n)$ . **Lernrate**  $\eta$  zu groß  $\Rightarrow$  Überschießen/Divergenz; zu klein  $\Rightarrow$  sehr langsam.

## 5 Integration & Interpolation

### 5.1 Newton-Cotes-Quadratur

Idee: Funktion durch ein Polynom über Stützstellen ersetzen und *dieses* exakt integrieren.

**Regeln auf  $[a, b]$ ,  $h = b - a$ ,  $m = \frac{a+b}{2}$ ; Gewichte für ein Panel**

Rechteck:  $h f(a)$  bzw.  $h f(b)$ ; Mittelpunkt:  $h f(m)$ ; Trapez:  $\frac{h}{2} [f(a) + f(b)]$ ;  
Simpson:  $\frac{h}{6} [f(a) + 4f(m) + f(b)]$ ; 3/8:  $\frac{3h}{8} [f_0 + 3f_1 + 3f_2 + f_3]$ ; Boole:  $\frac{2h}{45} [7f_0 + 32f_1 + 12f_2 + 32f_3 + 7f_4]$ .

### Genauigkeit & Exaktheitsgrad

Steigend: Rechteck ( $\mathcal{O}(h)$ , Grad 0)  $\rightarrow$  Mittelpunkt/Trapez ( $\mathcal{O}(h^2)$ , Grad 1)  $\rightarrow$  Simpson ( $\mathcal{O}(h^4)$ , Grad 3)  $\rightarrow$  Boole (Grad 5). **Exaktheitsgrad** = höchster Polynomgrad, der *exakt* integriert wird.

### Warum „bestes Rechteck“ und warum Simpson so gut

**Mittelpunkt** ist das beste Rechteck: er wertet in der Mitte aus, Über- und Unterschätzung der Krümmung heben sich auf  $\Rightarrow \mathcal{O}(h^2)$  statt  $\mathcal{O}(h)$ . **Simpson** ist durch Symmetrie sogar bis Grad 3 exakt, obwohl er nur eine Parabel anlegt.

### Zusammengesetzte Trapez-/Simpson-Gewichte richtig setzen

Bei zwei Teilintervallen ( $h = 1$ , Punkte  $f_0, f_1, f_2$ ):

Trapez:  $\frac{h}{2} [f_0 + 2f_1 + f_2]$  (der *innere* Punkt zählt 2 $\times$ , nicht  $f_1 + f_2$  zusammen!)

Simpson:  $\frac{h}{3} [f_0 + 4f_1 + f_2]$  (genau *ein*  $4f_1$ -Term – kein zusätzlicher Term).

$$I = \int_0^2 (x^2 + 1) dx, f(0) = 1, f(1) = 2, f(2) = 5$$

Exakt:  $\left[\frac{x^3}{3} + x\right]_0^2 = \frac{8}{3} + 2 = \frac{14}{3} \approx 4,667$ .

Trapez:  $\frac{1}{2} [1 + 2 \cdot 2 + 5] = \frac{1}{2} \cdot 10 = 5$  (Fehler  $\frac{1}{3}$ ).

Simpson:  $\frac{1}{3} [1 + 4 \cdot 2 + 5] = \frac{14}{3} = 4,667$  (Fehler 0).

Warum exakt?  $x^2 + 1$  hat Grad  $2 \leq 3$  = Exaktheitsgrad von Simpson; Trapez (Grad 1) erfasst die Krümmung nicht.

### 5.2 Lagrange-Interpolation

#### Lagrange-Polynom

Durch  $n+1$  Punkte  $(x_i, y_i)$  gibt es *genau ein* Polynom vom Grad  $\leq n$ :

$$p(x) = \sum_i y_i L_i(x), \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

$L_i$  ist 1 an  $x_i$  und 0 an allen anderen Knoten.

**Punkte**  $(0, 1), (1, 2), (2, 5)$

$$L_0 = \frac{(x-1)(x-2)}{2}, L_1 = -x(x-2), L_2 = \frac{x(x-1)}{2}.$$

$$p = 1 \cdot L_0 + 2L_1 + 5L_2 = \frac{(x-1)(x-2)}{2} - 2x(x-2) + \frac{5x(x-1)}{2} = x^2 + 1.$$

Probe:  $p(0) = 1, p(1) = 2, p(2) = 5 \checkmark$ ;  $p(\frac{1}{2}) = \frac{1}{4} + 1 = \frac{5}{4}$ .

**Runge-Phänomen – „stop at Simpson“**

Interpolation *hohen* Grades auf gleichmäßigen Knoten oszilliert stark am Rand (großer Fehler). Konsequenz: für Quadratur *nicht* den Grad erhöhen, sondern *zusammengesetzte* Regeln niedrigen Grades nutzen (Trapez/Simpson stückweise). Glatte Alternative: *Splines*.

### 5.3 Romberg & Monte-Carlo

**Romberg**

Trapezwerte für  $h, h/2, \dots$  und Richardson-Extrapolation eliminiert den  $O(h^2)$ -Term:  $R = \frac{4T(h/2) - T(h)}{3}$ .

**Monte-Carlo-Integration**

$\hat{I} = |\Omega| \frac{1}{N} \sum_{i=1}^N f(X_i)$ , Standardfehler  $SE = \frac{|\Omega| \sigma_f}{\sqrt{N}}$ . Da  $SE \propto N^{-1/2}$ : SE *halbieren*  $\Rightarrow N \times 4$ ; *zehnteln*  $\Rightarrow N \times 100$ .

**Fluch der Dimensionen**

Ein Gitter mit  $n$  Punkten pro Achse braucht in  $d$  Dimensionen  $n^d$  Punkte (exponentiell). Der MC-Fehler  $\mathcal{O}(1/\sqrt{N})$  ist *dimensionsunabhängig*  $\Rightarrow$  MC gewinnt bei großem  $d$ . Die *Mittelpunktregel* ist übrigens MC mit  $N = 1$  (ein Sample in der Mitte).

## 6 Modelltraining aus ersten Prinzipien

Lineare Regression  $\hat{y}(x) = wx + b$ , Verlust  $L(w, b) = \frac{1}{n} \sum_i (wx_i + b - y_i)^2$  (mittlerer quadratischer Fehler, **MSE**), minimiert per Gradientenabstieg.

**MSE & Varianz**

**MSE** = Mittel der *quadrierten* Residuen  $\frac{1}{n} \sum (\hat{y}_i - y_i)^2$ . **Varianz**  $\text{Var}(y) = \frac{1}{n} \sum (y_i - \bar{y})^2$  (Mittel der quadrierten Abweichungen vom Mittelwert).

**Varianz – die Abweichungen quadrieren!**

Für  $y = (2, 4, 6)$ :  $\bar{y} = 4$ . Die Abweichungen  $(-2, 0, 2)$  *summieren* sich zu 0 – das ist **nicht** die Varianz. Richtig:  $\text{Var} = \frac{1}{3} [(-2)^2 + 0^2 + 2^2] = \frac{1}{3} \cdot 8 = \frac{8}{3} \approx 2,67$ .

### Erster Loss = Varianz

Init  $w = 0$ ,  $b = \bar{y}$ : das Modell sagt überall  $\bar{y}$  voraus, also  $L = \frac{1}{n} \sum (\bar{y} - y_i)^2 = \text{Var}(y) = \frac{8}{3}$ . Der erste Loss *ist* die Varianz der Zielwerte. Ein gedruckter Loss  $\approx 100$  statt  $\frac{8}{3}$ :  $b$  falsch initialisiert oder *summiert* statt gemittelt ( $\approx n \cdot \text{Var}$ ). Ein Loss  $\approx 0$  beim Init: ein Bug (sollte nicht 0 sein).

### Overfit-Sanity-Check

Ein *überparametrisiertes* Modell kann den Loss auf einer *kleinen* Teilmenge leicht auf 0 treiben (es gibt unendlich viele Null-Loss-Lösungen). Gelingt das **nicht**, liegt es *nicht* am Optimierer, sondern an einem **Bug in Modell / Loss / Datenpipeline**. Das ist das numerische Prinzip: *einen Solver an einem Problem mit bekannter Lösung testen*.

### Trainingspraktiken $\leftrightarrow$ numerische Methode

- **Mini-Batch-SGD**  $\leftrightarrow$  *Monte-Carlo*: Gradient als Mittel über eine zufällige Stichprobe; Schätzfehler  $\propto 1/\sqrt{B}$  (Batch-Größe  $B$ ).
- **Mehrere Seeds, bester gewinnt**  $\leftrightarrow$  *Random Restarts*.
- **Quantisierung (niedrige Bit-Präzision)**  $\leftrightarrow$  *Präzisionswahl* im Gleitkomma (kleinstes Format, dessen Bereich & Genauigkeit reichen; Inferenz verträgt weniger Präzision als Training).

## 7 Anhang: weitere Foliensatz-Themen (kompakt)

Nicht im Notizen-Scope von Prüfung 03, aber Teil des vollen Foliensatzes:

### Differenzialgleichungen (Euler & Runge-Kutta)

**Expliziter Euler:**  $y_{n+1} = y_n + h f(t_n, y_n)$  (Ordnung 1). **Impliziter Euler:**  $y_{n+1} = y_n + h f(t_{n+1}, y_{n+1})$  (stabiler, für steife Probleme). **RK4:**  $y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$  (Ordnung 4). **Butcher-Tableau** kodiert Knoten  $c_i$ , Gewichte  $b_i$ , Matrix  $a_{ij}$ ; explizit  $\Leftrightarrow A$  streng untere Dreiecksmatrix. **Steifheit:** stark verschiedene Zeitskalen erzwingen bei expliziten Verfahren winzige  $h$ .

### Fourier & FFT

$F(\omega) = \int f(t)e^{-i\omega t}dt$  (Zeit  $\rightarrow$  Frequenz), Rücktrafo  $f(t) = \frac{1}{2\pi} \int F(\omega)e^{i\omega t}d\omega$ . Mit  $\cos(\omega t) = \frac{1}{2}(e^{i\omega t} + e^{-i\omega t})$  erzeugt jeder Kosinus zwei Peaks bei  $\pm\omega$  (*Interferenz* = Überlagerung). **DFT**  $X_k = \sum_n x_n e^{-i2\pi kn/N}$  ( $O(N^2)$ ), **FFT**  $O(N \log N)$ . **Nyquist** =  $f_s/2$ ; darüber *Aliasing*.

Lege beim Üben Wert auf: Trapez-/Simpson-Gewichte, Newton 2. Iteration (Nenner an neuer Stelle), Konsistenz vs. Stabilität, Varianz = quadrierte Abweichungen, Lagrange-Basispolynome.